

EXPERIMENT 10

Objective: Develop test cases for unit testing and integration testing

Unit Testing Test Cases:

Test Case Id	Test Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_E1	Add Book - Success	Verify that a book is added successfully when all required parameters are provided.	- Servlet container running. - Mock dependencies. - Database mock active.	1. Call the doPost method of BooksAdd . 2. Provide valid book details and a valid image.	The book should be added to the database, and the success message displayed.	As expected .	Pass
TC_E2	File Upload	Verify that the uploaded file is stored in the correct directory with the correct name.	- Servlet container running. - Mock file upload support.	1. Call the doPost method of BooksAdd . 2. Provide a valid file part and check the storage path.	File should be stored in the correct directory with the specified name.	As expected .	Pass
TC_E3	Delete Book - Success	Verify that a book is successfully deleted when a valid book ID is provided.	- Servlet container running. - Mock dependencies. - Database mock active.	1. Call the processRequest method of DeleteBookServlet . 2. Provide a valid book ID.	The book should be removed from the database, and the success message displayed.	As expected .	Pass

TC_E4	Sell Old Book - Success	Verify that a book is successfully added as "Old" when all parameters are valid.	- Servlet container running. - Mock dependencies. - Database mock active.	1. Call the <code>processRequest</code> method of <code>SellOldBookServlet</code> . 2. Provide valid parameters (e.g., book name, author, price).	Book is added with the category "Old", status "Active", and success message displayed.	As expected .	Pass
TC_E5	Process Valid Order	Verify <code>processRequest()</code> processes a valid order correctly.	Valid user session, valid cart data.	1. Mock request with valid user and order details. 2. Call <code>processRequest()</code> . 3. Verify redirection to success page.	Redirect to <code>order_success_page.jsp</code> , order details saved successfully.	As expected .	Pass

Integration Testing Test Cases:

Test Case Id	Test Name	Objective	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC_I1	End-to-End Order Processing	Verify end-to-end order processing from adding items to placing an order.	Valid user session, cart items present.	1. Add items to cart. 2. Proceed to checkout. 3. Select payment type. 4. Place order. 5. Verify order summary.	Order is placed, order summary is displayed, and cart is cleared.	As expected .	Pass

TC_I2	Cart Retrieval Integration	Ensure <code>CartDao</code> correctly retrieves cart items for a user and <code>OrderServlet</code> processes them.	User has items in the cart.	1. Mock request with valid user.2. Call <code>processRequest()</code> .3. Verify cart items are retrieved and processed.	Cart items are retrieved and order is processed correctly.	As expected .	Pass
TC_I3	Database Save Integration	Verify <code>BookOrderDao</code> saves order to database and <code>OrderServlet</code> handles the response.	Database is connected and operational.	1. Mock request with valid order details.2. Call <code>processRequest()</code> .3. Verify order is saved in the database.	Order is saved in the database and the success page is displayed.	As expected .	Pass

EXPERIMENT 11

Objective :Develop test cases for various white box and black box testing techniques.

Testing Type	Test Case Description	Expected Output	Actual Output	Pass/Fail
White Box	Verify <code>addBook()</code> function handles both valid and invalid inputs through all code paths.	Valid inputs add the book successfully; invalid inputs trigger appropriate error handling.	As expected	Pass
White Box	Ensure <code>deleteBook()</code> handles different scenarios like invalid IDs and database failures.	Book is deleted for valid IDs; invalid IDs or failure scenarios trigger error messages.	As expected	Pass
White Box	Validate data flow in <code>checkout()</code> from cart to payment processing.	Payment is processed successfully, and order is confirmed; errors are handled for invalid data.	As expected	Pass
Black Box	Test adding a book with valid and invalid data.	Book is added successfully for valid data; error messages for invalid data.	As expected	Pass
Black Box	Verify user login functionality for different credential inputs.	Successful login for valid credentials; error messages for invalid credentials.	As expected	Pass
Black Box	Test cart and checkout process for multiple items and error conditions.	Items are added to cart, and checkout completes without issues; errors are handled gracefully.	As expected	Pass